RedByte

FPGA EDUCATIONAL IDE

Product Manual

Version 1.3 - July 2026

Target Platform: Digilent Basys 3 (Artix-7 XC7A35T)

Toolchain: AMD Vivado ML Standard Edition

Attribution: Connor Angiel

DOCUMENT CONTROL

Document ID	RB-MAN-001
Version	1.3
Date	2026-07-27
Release posture	Stable Preview - Browser-E0; no Vivado, bitstream, board, or classroom-reliability claim
PDF artifact	Generated and visually inspected from this synchronized print source during stable-preview closeout
Classification	Product Reference
Repository	`redbyte-ui` monorepo
Primary Package	`packages/rb-apps`
Target Board	Digilent Basys 3 (Artix-7 XC7A35T)
Part Number	`xc7a35t-1cpg236-1`
License	RedByte Proprietary License (RPL-1.0)

CONVENTIONS

NOTE

Supplementary information or clarification.

WARNING

Important information that may affect results.

CAUTION

Action that may cause data loss or errors.

TIP

Best practice or recommended approach.

Table of Contents

1. Introduction

This manual is the canonical product reference for RedByte, a browser-based digital logic design, simulation, verification, and FPGA hardware-preparation environment. It describes the product as it exists, explains its operating model, documents each major surface and workflow, and provides reference material for all audiences.

Students beginning their first project should read Sections 2 through 6, then follow the workflow sections (8 through 11) in order. **Instructors and graders** should additionally read Section 13 for submission and review workflows. **Evaluators and developers** will find the appendices and glossary useful as standing reference material.

2. Product Overview

2.1 What RedByte Is

RedByte is a deterministic FPGA learning and project-building environment that runs entirely in a web browser. It provides a unified workspace where users create supported combinational and sequential projects, verify behavior with authored testbench checks, map top-level ports to physical FPGA board pins, export a complete file set for AMD Vivado, and record proof when Vivado or board behavior matters.

RedByte targets the **Digilent Basys 3** development board (Xilinx Artix-7 XC7A35T FPGA). It reduces the fragmented workflow of separate schematic editors, simulation tools, and manual constraint-file authoring, but it does not replace Vivado, perform timing closure, or guarantee arbitrary HDL/hardware success. Vivado remains the tool that synthesizes, implements, generates bitstreams, and programs hardware.

2.2 The Problem RedByte Solves

Digital-logic work traditionally spans a schematic or HDL editor, simulator, constraint editor, and synthesis toolchain. RedByte keeps supported browser authoring, verification, Basys3 mapping, package generation, and recovery in one coherent workspace while preserving the external Vivado and hardware proof boundary.

2.3 Core Capabilities

CAPABILITY	DESCRIPTION
Circuit Design	Visual drag-and-drop canvas for supported gates, boundary I/O, native registers, legacy/theory sequential elements, and reusable blocks. The current palette has no generic Clock card: FPGA clock intent uses the Basys3 `CLK100MHZ` board resource and pure simulation can use an automatic internal clock.
Deterministic Simulation	Tick-based engine using topological-sort evaluation. Same circuit + same inputs = same outputs, every time, on every machine.
Truth Table Verification	Automated testing against expected truth table vectors with per-row pass/fail reporting. Supports combinational and sequential schedules.
Hardware Pin Mapping	Dedicated interface for assigning circuit ports to Basys 3 physical pins — switches, LEDs, buttons, 7-segment display, clock.
Vivado Export	Complete export pipeline generating synthesizable VHDL, pin constraints (XDC), and a VHDL testbench in a single ZIP.
Project Import	Import VHDL, XDC, or RedByte project archives with explicit fidelity reporting (Full, Reconstructed, or Partial).
Submission Packaging	Deterministic, tamper-evident submission export with SHA-256 integrity hashing for instructor review.

2.4 Design Philosophy

PRINCIPLE	MEANING
Determinism by Design	Every simulation tick is reproducible. No hidden randomness, no race conditions, no undefined behavior.
One Truth, Many Views	The circuit is the single source of truth. Every surface is a projection of the same data.
Truth over Simplification	Gates have propagation delay. Circuits stabilize over ticks. Students learn correct mental models.
Local-First Operation	All computation happens in the browser. No server, no account, no data leaves the machine.

2.5 Stable Preview Authority

The stable-preview source unifies the student flow as **Project → Design Edit / Live → Verify Scenario / Replay / Optional Checks → Map Pins → Export**, with **Import / Recover** as a separate recovery utility. The preserved v2B candidate is `a5b67274f3c43820e89d538cbf2171256fef3759`; the integrated product baseline for this manual is `66f901ff13b6ddd0a0a73a4328a95c4df5274886`. Project explains the loaded work; Design keeps the circuit grid dominant; Verify owns named scenarios, authored stimulus, optional checks, sequential policy, waveform evidence, and Replay; Map Pins owns the semantic signal-to-resource projection; Export separates structural, verification-trust, and download state; Import is manifest-first and preserves exact scalar/vector-bit identities.

Named sequential policy remains browser-local and outside portable `RBProject`, but it is not package-neutral. Authored rows and policy are materialized into one shared execution-vector sequence consumed by runtime Verify, bring-up expectations, and generated `testbench.vhd` together with the resolved clock/schedule projection. Auto `runCycles`, automatic reset behavior, resolved clock data, starting level, and authored stimulus may change package bytes, stale Export, and invalidate a prior receipt. UI status, waveform, and Compare-result objects do not generate bytes. This is Browser-E0/software-artifact behavior, not Vivado or board proof.

STABLE-PREVIEW BOUNDARY

The completed v2B evidence includes 157 focused tests plus workspace typecheck, IDE CSS audit, unified build, and browser evidence at 1366×768 and 1440×900 under Node 20.19.0 / pnpm 10.24.0. Stable-preview closeout additionally repaired stale invalid-IR fixtures and explicit-scenario authority. This is Browser-E0/local package evidence, not Vivado execution, bitstream generation, board programming, physical observation, or unsupervised classroom reliability. Guided 4-bit, Mapping Assistant v2, RegisterBus, and StateBank execution remain deferred.

3. Intended Users and Usage Contexts

AUDIENCE	PRIMARY CONTEXT	KEY ACTIVITIES
Students	IdeApp	Design circuits, verify truth tables, map pins, export for Vivado, submit lab work.
Instructors / TAs	Submission Inspector (via Project surface)	Review submissions, verify integrity, inspect gate verdicts, grade with full diagnostics.
Self-Learners	IdeApp	Open-ended circuit exploration using the full IDE.
Evaluators	All contexts	Assess product capabilities, review workflows, evaluate platform for adoption.

Application Contexts

CONTEXT	PURPOSE	SUBMISSION SYSTEM	DIAGNOSTIC DETAIL
IdeApp	Five-stage lab IDE plus the Import / Recover utility	Yes	Student-appropriate (simplified)
SubmissionInspectorApp	Instructor grading tool (inspector functionality delivered via Project surface)	Read-only review	Full (hashes, verdicts, tamper detection)

4. Core Concepts and Operating Model

4.1 The Canonical Workflow

RedByte organizes work into the product spine: **Project → Design → Verify → Map Pins → Export → Vivado → Program Board → Observe**. The five horizontal student stages end at Export. Import / Recover is a separate utility for bringing external HDL or previously exported projects into the environment; it is not a sixth stage.

4.1.1 Draft, Trusted, and Proven States

STATE	WHAT IT MEANS
Draft design	A circuit exists, but it may not have current testbench, mapping, or export proof.
Simulated	The design has been observed in RedByte simulation; this is useful inspection, not a pass/fail claim.
Testbench configured	Stimulus and expected output checks exist for the current design intent.
Compare passed	Current observed outputs match current expected outputs. This is the Verify proof needed for trusted handoff.
Pins mapped	Required top-level ports are assigned to board resources/package pins.
Draft export	A structurally valid Vivado package can be generated or downloaded, but proof is missing or stale.
Trusted export	Current Compare PASS, current mapping, and current export bundle all describe the same project state.
Vivado built	Vivado synth/implementation/bitstream completed for the exported project.
Board programmed	A Basys 3 board was programmed with the generated bitstream.
Board observed	Physical behavior was recorded against an agreed observation procedure.

4.2 Simulation Model

RedByte's simulation engine operates on discrete **ticks**. Each tick, the engine evaluates every node in **topological order** — nodes whose inputs depend only on already-evaluated nodes are evaluated first.

This ordering guarantees deterministic, race-free signal propagation.

> **NOTE**
>
> For **combinational circuits**, evaluation typically stabilizes within one or two ticks. For **sequential circuits**, the engine detects clock edges by comparing the current clock value against the previous tick's value. A rising edge (0→1) triggers data capture in flip-flops.

In manual/custom mode, each authored row is one settled sample and drives the resolved clock input. Only low-to-high advances rising-edge state; repeated high, high-to-low, repeated low, and flat-low stimulus hold it. An authored falling transition is valid stimulus, not falling-edge-triggered capture. The saved `activeEdge` field is normalized to `rising` in this RC; it does not enable falling-edge capture. Manual/custom execution adds no hidden reset. Auto materializes cycle 0 and the selected run cycles; every Auto report row and generated VHDL assertion is post-rising-edge. Automatic reset, when selected, is explicit in cycle 0 and later deassertion rather than a hidden runtime prelude.

4.3 Verification Model

Verification compares actual circuit outputs against expected values defined in **test vectors**. RedByte automatically selects the correct verification schedule:

SCHEDULE	CIRCUIT TYPE	BEHAVIOR PER VECTOR ROW
Combinational	No flip-flops	Apply inputs → one tick → read outputs
Auto board clock	Contains flip-flops driven by the board clock	Start at cycle 0; materialize the greater of run-cycle count, authored-row count, and one, including automatic reset rows when applicable → sample every row post-rising-edge
Manual/custom	Contains authored clock/control stimulus	Drive authored clock → settle one row → capture only on low-to-high

4.4 Architecture Layers

Layer E — UX Shell	Five student workflow stages plus the separate Import / Recover utility. No internal diagnostic language in the default view.
Layer D — Submission	Deterministic ZIP bundling with integrity hashes. Tamper detection and gate verdicts.
Layer C — Vivado Adapter	VHDL generation, XDC constraints, testbench generation, sequential analysis.
Layer B — Verification	Truth table comparison, pass/fail per vector, failing node identification.
Layer A — Logic Core	Deterministic circuit simulation. Gate evaluation, topological sort, signal propagation.

Each layer depends only on layers below it. The Logic Core has no upward dependencies. The UX Shell orchestrates calls to all lower layers on user actions.

4.5 Import Fidelity Model

LEVEL	NAME	SOURCE	WHAT IS PRESERVED
1	Full	RedByte project archive (`.rbproj.json`)	All state: circuit, layout, vectors, probes, mappings.
2	Reconstructed	Supported structural VHDL or RedByte-generated concurrent-assignment subset	Supported topology. Auto-layout. Manifest required for vectors, mapping, and metadata.
3	Partial	Arbitrary behavioral/process HDL or unsupported constructs	I/O ports only, or blocked.

5. Getting Started

5.1 System Requirements

RedByte runs in any modern web browser with JavaScript enabled (Chrome, Edge, or Firefox recommended). For FPGA synthesis, **AMD Vivado ML Standard Edition** (2024.1+) and a USB connection to the Basys 3 board are required.

5.2 Development Installation

```
git clone <repository-url>
  cd redbyte-ui-genesis-main
  corepack pnpm install --frozen-lockfile
  corepack pnpm run dev
```

The development server starts at `http://localhost:5173`.

5.3 Starter Examples

EXAMPLE	DESCRIPTION	TYPE
Signal Tour	Four-wire passthrough (SW→LD)	Combinational
Logic Gates	AND, OR, XOR with 2 switches, 3 LEDs	Combinational
Half Adder	Sum and carry from two inputs	Combinational
Full Adder	1-bit addition with carry-in/out	Combinational
Two-Bit Counter	2-bit counter with clock and reset	Sequential

5.4 Quick Walkthrough: AND Gate to Vivado Export

1. Open RedByte and navigate to the **Design** surface.
2. Drag two **Switch** nodes and one **AND** gate from the palette onto the canvas.
3. Drag one **Lamp** node onto the canvas.
4. Wire each Switch output to an AND gate input. Wire the AND output to the Lamp input.
5. Navigate to **Verify**. Add expected rows or use defaults. Click **Run Compare**.
6. Navigate to **Map Pins**. Assign switches to SW0/SW1 and the lamp to LD0.
7. Navigate to **Export**. Click **Download Package**.

> **Result:** A ZIP file containing `top.vhd`, `top.xdc`, `testbench.vhd`, automation scripts, and documentation files. RedByte may label this as a draft package or a trusted handoff depending on whether Compare, mapping, and export evidence are current. See Section 11.1 for the complete file list.

6. Workspace and Interface Overview

6.1 Global Shell

The IDE shell consists of three persistent regions visible on every surface:

REGION	LOCATION	CONTENT
Top Bar	Top	Product/project identity, board target, save state, Import / Recover utility, and Help.
Stage Navigator	Below top bar	Five horizontal stages: Project, Design, Verify, Map Pins, Export. Import remains an unnumbered utility.
Main Content	Center	The active surface's primary work object and state-owned action.

6.2 Navigation and Status

Surface switching is explicit through the horizontal stage navigator and state-owned CTAs. Pills are reserved for semantic states such as PASS, FAIL, stale, and blocked; ordinary information is shown as headings, rows, labels, or sentences.

7. Detailed Surface Reference

7.1 Project Surface

ASPECT	DETAIL
Purpose	Action-first entry plus a useful engineering overview of the loaded student's project.
When to Use	Session start, project creation, readiness review, metadata changes.
Main Area	Textual engineering overview: identity, Design, Verify, Map Pins, Export, blocker, and one recommendation.
Controls	Edit identity, follow one recommended next action, open the owning Design/Verify/Map Pins/Export row, or use Change Project with replacement guards.
Outputs	Updated metadata and authoritative workflow summaries; Project does not duplicate mapping or package editors.
Student View	Project identity, a read-only live circuit snapshot, Design/Verify/Map Pins/Export summaries, one recommendation, and state-owned actions such as **Open Export**. Export's technical evidence dialog contains secondary hashes/provenance.

7.2 Design Surface

ASPECT	DETAIL
Purpose	Circuit construction via drag-and-drop gate placement and wiring.
When to Use	Whenever the circuit needs to be created or modified.
Main Area	Stable 200–220px library, dominant circuit canvas with direct Select/Wire/Undo/Redo/Fit/Zoom/View controls, and stable 240–280px inspector.
Canvas modes	**Edit** owns structural authoring, **Live** shows exploratory propagation without creating saved Verify evidence, and read-only **Replay** is enabled only for a recorded Verify run.
Controls	Place components, wire explicit ports, select/move/delete, undo/redo (100 levels), and use keyboard-reachable port targets.
Geometry	Circuit grid: 63.1% at 1366px and 65.0% at 1440px; 62% floor passed, strategic 70% target unmet. Sparse port targets are at least 24×24px; dense targets are at least 32×24px (current 32×36px).
Palette	A Common group starts with AND/OR/XOR/NOT/Register1/INPUT/OUTPUT, followed by AND/OR/NOT/NAND/NOR/XOR/XNOR, five 3-input gates, Register1/RegisterBus/StateBank, legacy/theory sequential elements, INPUT/OUTPUT/Ground, FullAdder, custom components, and separate Basys3 board resources. Generic Clock is not a current palette card.
Registry boundary	27 direct primitive registrations plus 4 composite registrations = 31 runtime registry additions. Runtime registration does not guarantee student-palette visibility.
Wiring Rules	Output→Input valid. Fan-out allowed. Self-loops rejected. One connection per input port.

ASPECT	DETAIL
Health Indicator	"Circuit OK" or "Issues found" (floating outputs, missing I/O).

TIP

Compare failures separate **Edit expected** from **Inspect Design**. Structural preflight failures use **Open Design** before Compare can run.

7.3 Verify Surface

ASPECT	DETAIL
Purpose	Author stimulus, run deterministic simulation, inspect waveform or circuit replay, and add optional expected-output checks.
When to Use	After building or modifying a circuit, before hardware mapping or export.
Main Area	Visual scenario cards summarize document type, event count, optional check count, timing cycles, and signal preview; Scenario, Replay, and Checks remain lenses around one run authority.
Controls	Use **Run simulation**, edit combinational rows or sequential timeline cells, inspect mismatch, choose **Edit expected**, **Inspect Design**, or structural **Open Design**, then rerun.
Sequential policy	Each named document owns execution override, run cycles, active edge, reset behavior, source type, execution model, resolved signal identity, and starting level where available. Auto board clock, manual pulses, and custom pattern are distinct choices.
Sequential execution	One materialized vector sequence drives runtime, bring-up expectations, and testbench generation. Manual/custom captures only on authored low-to-high; Auto materializes cycle 0/run cycles/reset and samples every row post-rising-edge. No hidden runtime reset prelude is added.
Verify evidence currentness	`current`, `missing`, `stale`, and `failed` are distinct upstream evidence states, not Export `verificationTrust` values. Observe-only traces do not support trusted classification.
Waveform	Run transport and current/stale verdict stay adjacent to readable trace evidence; lanes use 36×36px interaction targets and labels remain at least 13px.

ASPECT	DETAIL
Hints	14 diagnostic conditions evaluated on FAIL; matching fact-grounded hints displayed. Sequential circuit banner when DFF detected.
Freshness	A semantic Design fingerprint excludes node position/layout and project identity text. Topology, node type, logical I/O, and testbench-authority changes revoke current evidence.

AUTHORSHIP AND DESIGN REPAIR

Named testbench identity, active document, cases, stimulus, expected values, sequential steps, and sequential execution policy are authored browser-workspace state. An Unset expected cell means no check for that output on that row; simulation may still record it. A stimulus-only document remains stimulus-only through compatible Design edits, undo, and redo; RedByte does not manufacture checks from observed outputs. Layout-only node moves preserve authorship and current evidence. Compatible truth-affecting Design edits preserve or rekey the same document but clear current Compare/waveform authority; structural breaks report **DESIGN BLOCKED** until repaired and rerun. Browser-local save/autosave/restore stores named `scenarios` and the active document beside portable `RBProject` JSON; this sidecar is not a new portable project field or proof of cross-browser/archive transfer. Its authored rows and policy can still change the shared materialized execution vectors and resolved clock/schedule projection used by generated `testbench.vhd`, package freshness, and receipt authority.

7.4 Map Pins Surface

ASPECT	DETAIL
Purpose	Map circuit ports to physical Basys 3 pins. Required before export.
When to Use	After circuit is designed and verified.
Main Area	Progress plus grouped mapping table; stable selected-signal editor; secondary non-assignment Basys 3 reference.
Controls	Select row **Assign**/**Edit mapping**/**Resolve**, choose a compatible Basys3 resource, inspect the package-pin/XDC consequence, and use **Save assignment**. The board graphic is reference-only.
Projection	Logical signal, direction, artifact port, board resource, package pin, I/O standard, exact XDC line, required state, and conflict state remain one coherent semantic row across Map Pins and Export.
Boundary	Proves saved mapping for XDC generation, not Vivado build, programming, or board behavior.
Available Pins	16 switches, 16 LEDs, 5 buttons, 7-segment display (7 cathodes + 4 anodes + DP), CLK100MHZ.

7.5 Export Surface

ASPECT	DETAIL
Purpose	Validate readiness, preview generated artifacts, produce Vivado Kit ZIP.
When to Use	After design, verification, and complete pin mapping.
Main Area	**What should I submit?**, blocked/draft/ready handoff decision, and stable file list/selected preview for meaningful packages.
Entry	State-owned actions elsewhere use **Open Export** to route package work into this stage rather than duplicating Export controls.
Primary CTA	**Download Package** is reserved for a trusted current handoff. A structurally valid but untrusted project exposes the separate **Download draft** action.
Files and evidence	The stable file browser selects exact generated content for preview/copy/download. **Open technical evidence** opens a separate diagnostics/provenance dialog.
Trust and receipt	Structural `blocked` / `downloadable`, `verificationTrust` `unverified` / `draft` / `trusted`, and action `not-downloaded` / `downloaded` stay separate. Verify evidence currentness remains upstream. The current receipt binds source fingerprint, project/Verify hashes, mapping currentness, download kind, trust state, and SHA-256 to the exact package.
Sequential testbench	Runtime, bring-up expectations, and testbench generation share materialized execution vectors plus the resolved clock/schedule projection. Auto `runCycles` and automatic reset can change bytes; every Auto assertion is post-rising-edge. Manual/custom assigns authored clock values and settles without the Auto scaffold.
Manifest agreement	The generated manifest embeds the exact generated `top.vhd` and `top.xdc`; preview, semantic mapping projection, ZIP, and manifest agree.

ASPECT	DETAIL
Scaffold Warnings	HDL/XDC port mismatch warnings for projection-only exports are informational, not blocking.

7.6 Import Surface

ASPECT	DETAIL
Purpose	Import external HDL, constraint files, or RedByte project archives.
Sequence	Horizontal Upload → Review → Apply. ZIP, **Paste HDL**, conditional **Paste XDC**, and samples are source choices inside Upload, not workflow tabs or stages.
Main Area	Candidate identity/design/mapping/fidelity review followed by explicit replacement confirmation; Cancel preserves current work.
Fidelity	Full (manifest-first project archive), Reconstructed (supported structural or RedByte-generated concurrent-assignment VHDL), Partial/blocked (arbitrary behavioral or unsupported HDL).
Identity	Scalar and vector-bit ports such as `SW[1]` and `LED[1]` retain exact identity through Review, Apply, mapping, and re-export.
Submission Detection	Identifies RedByte submission ZIPs and reports integrity status.

WARNING

Supported RedByte-generated concurrent-assignment `top.vhd` can reconstruct the supported graph, but this is not a lossless restore. Use the embedded manifest for layout, authored Verify documents, mapping, and metadata. Loose sibling HDL/XDC cannot override a valid manifest.

8. Circuit Design Workflow

8.1 Placing Components

1. Open the **Design** surface.
2. Locate the desired component in the left palette (use search to filter by name).
3. Drag the component onto the canvas and release to place.
4. Repeat for all required components.

8.2 Wiring Components

1. Click on an **output port** (right side of a gate, or Switch output).
2. Drag to the target **input port** (left side of a gate, or Lamp input).
3. Release on the target port.

> **Result:** A wire appears. The signal flows from source to destination. Invalid connections are rejected with feedback.

8.3 Sequential Elements

For the supported stable-preview sequential path:

1. Place **Register1** on the canvas.
2. Connect the **D** (data) port to the source signal.
3. For FPGA intent, connect **CLK** to a top-level clock input created from the Basys3 `CLK100MHZ` board resource. Pure simulation can use RedByte's automatic internal simulation clock; the current Design palette has no generic Clock card.

4. Optionally connect **EN** (enable) and **RST** (reset) ports.
5. Connect the **Q** output to downstream logic or output indicators.

> **NOTE**
>
> Register1 captures input on the **rising edge** of the clock. Between rising edges, output Q holds its value.

Stable-preview boundary. Register1 captures D on the rising clock transition and holds Q between rising edges. An authored high-to-low transition is valid stimulus but must hold rising-edge state. RegisterBus, StateBank, falling-edge capture, multi-clock designs, active-low reset, and unsupported register modes remain blocked.

9. Verification Workflow

1. Navigate to the **Verify** surface.
2. Choose or create the named scenario, then review or author stimulus rows. Expected-output checks are optional.
3. For a sequential document, review its saved execution mode, run cycles, active edge, reset behavior, source type, execution model, resolved clock identity, and starting level. Choose automatic board clock, manual pulses, or custom pattern deliberately. In manual/custom mode, only low-to-high advances rising-edge state; falling and flat/repeated levels hold it.
4. Click **Run simulation** and inspect waveform or circuit Replay.
5. If validation is required, add expected-output checks and rerun. Blank check cells are not assertions.
6. Review results: ✓ per passing row, ✗ per failing row.
7. For failures, choose **Edit expected** when the saved check is wrong or **Inspect Design** when the circuit is wrong. Structural preflight failures expose **Open Design**.
8. Apply the correct repair and rerun the scenario.

> **Result:** A successful zero-check run reports **Simulation complete** and **No checks configured**; it does not claim PASS. A current `PASS` applies only when configured checks pass for the same current design and scenario. Missing, stale, and `FAIL` evidence cannot support Export `verificationTrust: trusted`.

Verification Determinism

Results are deterministic. Running the same circuit with the same vectors produces the same results every time, on every machine. The Verify surface displays a deterministic run hash to confirm reproducibility.

10. Hardware Mapping and Board Preparation

1. Navigate to the **Map Pins** stage.
2. Choose **Assign**, **Edit mapping**, or **Resolve** for one signal-table row.
3. Use the selected-signal **Basys3 resource** selector, review the package-pin/XDC consequence, and choose **Save assignment**. The smaller board graphic is reference-only.
4. For sequential FPGA designs with a top-level clock port, assign **CLK100MHZ** (pin W5).
5. Repeat until all required rows are assigned and no conflicts remain.
6. Confirm the semantic preview agrees on logical identity, artifact port, board resource, package pin, I/O standard, and exact generated XDC line.

> **NOTE**
>
> The export pipeline automatically emits `CLOCK_BUFFER_TYPE NONE` for switch and button inputs. This prevents Vivado from inserting illegal BUFG paths on non-clock-capable pins.

11. Vivado Export and External Tool Workflow

11.1 Generated Files

FILE	TYPE	PURPOSE
`top.vhd`	Design Source	Synthesizable VHDL top-level entity.
`top.xdc`	Constraints	Pin assignments with LVCMOS33 I/O standard.
`testbench.vhd`	Simulation Source	VHDL testbench from the shared materialized execution vectors plus resolved clock/schedule projection.
`vivado_import.tcl`	Automation	Tcl script for automated Vivado project creation.
`program_and_test.tcl`	Automation	Tcl script for board programming and test.
`README.txt`	Documentation	Quick-start instructions for the Vivado workflow.
`BRINGUP.md`	Documentation	Detailed board bring-up guide.
`EXPECTED_IO.json`	Metadata	Machine-readable I/O expectations for automated verification.
`project.rbproj.json`	Project	RedByte project snapshot for round-trip import.

Runtime Verify, bring-up expectations, and generated `testbench.vhd` consume the same materialized execution vectors plus the resolved clock/schedule projection. Auto board-clock testbenches use a

free-running `clock_gen` and `CLK_HALF_PERIOD`, then wait for a rising edge before every materialized row assertion, including cycle 0. Auto `runCycles` and automatic reset behavior may change the vector sequence and package bytes; automatic reset is materialized rather than injected as a hidden runtime prelude. Manual/custom testbenches omit that scaffold, assign authored clock values, and sample after the deterministic settle interval. This is Browser-E0/software-artifact authority, not proof that Vivado executed the testbench.

11.2 Importing into Vivado

1. Use **Download Package** for a trusted current handoff, or deliberately choose the separate **Download draft** action for an untrusted but structurally buildable package; then extract the ZIP.
2. Open Vivado → **Create Project** → RTL Project.
3. Add `top.vhd` as Design Source. Add `top.xdc` as Constraints.
4. Select part: `xc7a35t-1cpg236-1` (or board "Basys3").
5. **Run Synthesis** → **Run Implementation** → **Generate Bitstream**.
6. Open Hardware Manager → connect board → **Program Device** with the generated `.bit` file.

> **Result:** The FPGA is programmed. Toggle physical switches and observe LEDs to verify the design matches simulation.

11.3 Common Vivado Errors

ERROR	LIKELY CAUSE	RESOLUTION
"Port X not found in entity"	Port name mismatch	Re-map in Map Pins and re-export.
"Multiple drivers on net"	Combinational loop	Fix the loop in Design, re-verify, re-export.
"Timing not met"	Excessive logic depth	Simplify combinational path or add pipeline registers.

11.4 Official References Used for Vivado Truth

- AMD Vivado Design Suite User Guide: Using Constraints (UG903), Timing Constraints: https://docs.amd.com/r/en-US/ug903-vivado-using-constraints/Timing-Constraints
- AMD Vivado Design Flows Overview (UG892/UG895), Project Mode Tcl Commands: https://docs.amd.com/r/2023.2-English/ug892-vivado-design-flows-overview/Using-Project-Mode-Tcl-Commands
- AMD Vivado Programming and Debugging (UG908), Programming the Hardware Device: https://docs.amd.com/r/en-US/ug908-vivado-programming-debugging/Programming-the-Hardware-Device
- Digilent Basys 3 Master XDC for board labels, package pins, and the 100 MHz clock: https://github.com/Digilent/digilent-xdc/blob/master/Basys-3-Master.xdc

12. Import and Reuse Workflows

12.1 Full-Fidelity Re-import

Choose the `.rbproj.zip` archive in Import's Upload step. The embedded manifest is authoritative, its generated HDL/XDC projection must agree with the package, and loose sibling files cannot override it. Review the detected project, then use the explicit Apply confirmation; Cancel preserves current work.

12.2 Structural VHDL Import

Inside Import's Upload step, choose **Paste HDL**, paste structural VHDL with component instantiation, and choose **Parse HDL**. The parser maps 37 HDL name variants (e.g., `and2`, `AND`, `and_gate`) to 9 RedByte node types. Review the candidate, choose **Review replacement**, then use the explicit **Confirm replacement** action; Cancel preserves the current project.

Supported RedByte-generated concurrent-assignment `top.vhd` also reconstructs the supported graph. It remains reconstructed, not lossless: layout, authored Verify documents, mapping, and metadata require the manifest. Arbitrary behavioral/process HDL remains ports-only or blocked. Scalar/vector-bit identities such as `SW[1]` and `LED[1]` must survive Review, Apply, mapping, and re-export unchanged.

12.3 XDC Constraint Import

Inside the same Upload step, choose **Paste HDL** and parse the structural source first; then choose **Paste XDC**, paste constraints, and choose **Parse XDC**. Review and confirm the combined candidate before replacement.

13. Submission, Review, and Instructor Workflows

13.1 Student Submission

1. Complete circuit design and achieve a `PASS` verification.
2. Navigate to **Export**, or use a state-owned **Open Export** action.
3. Read **What should I submit?**, then choose **Download Package** for a trusted current handoff. Use **Download draft** only when the course accepts an explicitly untrusted package.
4. Inspect the nine generated files in the file browser; use **Open technical evidence** only for diagnostics/provenance.
5. Check that the receipt's download kind, trust state, source fingerprint, project/Verify hashes, mapping currentness, and SHA-256 describe the exact downloaded package.
6. Upload the downloaded archive to the institutional LMS.

The submission is **deterministic**: same project state always produces the same bytes. This enables tamper detection.

13.2 Instructor Review

1. Open the **Submission Inspector**.
2. Import the student's submission archive.
3. Review: gate verdict, pass/fail per requirement, integrity status, hash verification, tamper detection.

13.3 Integrity Properties

PROPERTY	MECHANISM
Deterministic packaging	Same state → identical output bytes.
Content-addressed hashing	SHA-256 hash per file in manifest.
Tamper detection	Any file modification invalidates checksums.
Optional signing	Ed25519 signatures. Unsigned accepted but marked.

13.4 Gate Verdicts

CONTEXT	DISPLAY
Student view (IdeApp)	"Ready to submit" / "Submission needs attention"
Instructor view (Inspector)	Raw PASS/FAIL with detailed breakdown

14. Files, Packages, and Generated Outputs

14.1 Vivado Kit ZIP

```
vivado-kit.zip
├── top.vhd                 # Synthesizable VHDL
├── top.xdc                 # Basys 3 pin constraints
├── testbench.vhd           # Simulation testbench
├── vivado_import.tcl       # Automated Vivado project creation
├── program_and_test.tcl    # Board programming and test script
├── README.txt              # Quick-start instructions
├── BRINGUP.md              # Board bring-up guide
├── EXPECTED_IO.json        # Machine-readable I/O expectations
└── project.rbproj.json     # RedByte project snapshot
```

The Vivado-kit `project.rbproj.json` is a generated package projection. It embeds the exact generated `top.vhd` and `top.xdc` content used by the same package; Import validates that authority before a full restore. Named testbench documents remain browser-local and outside portable `RBProject`, while their authored rows and policy may still change the materialized execution vectors and resolved clock/schedule projection used by generated `testbench.vhd`.

14.2 Submission Package

```
submission.rbproj.zip
├── rb-project.json       # Complete project state
├── verify-ledger.json    # Verification results
├── manifest.json         # File hashes / integrity
└── [metadata files]      # Gate verdicts, run counts
```

15. Troubleshooting and Error Resolution

15.1 Circuit Design Issues

SYMPTOM	CAUSE	RESOLUTION
Wire will not connect	Invalid direction	Source must be output port, target must be input port.
"Issues found" health	Floating outputs / missing I/O	Connect all outputs; add INPUT/OUTPUT nodes for hardware ports.
Flip-flop output unchanged	No effective clock activity	For FPGA intent, connect a top-level clock and map it to `CLK100MHZ` (W5). For simulation-only intent, confirm Verify reports the automatic internal clock policy or author the required manual control activity.

15.2 Verification Issues

SYMPTOM	CAUSE	RESOLUTION
All vectors fail	Wiring error	Trace signal paths in Design surface.
Sequential test fails	Missing clock/control activity	Confirm the top-level board clock mapping or Verify's automatic/internal/manual clock policy, then rerun.
Outputs advance while a manual/custom clock stays flat	Invalid execution evidence: state advanced without an authored low-to-high transition	Do not use the run to authorize Export. Repair the stimulus or product regression, then rerun Compare.
Stale indicator	Circuit modified	Re-run verification.
Testbench remains after Design edit but PASS/waveform disappears	Design behavior changed, so prior evidence is archival	Repair any Design blocker and rerun Compare; authored cases were retained.

15.3 Export Issues

SYMPTOM	CAUSE	RESOLUTION
Export BLOCKED	Incomplete prerequisites	Complete design, verify, and pin mapping.
"HDL ports missing in XDC"	Scaffold warning	Expected for simple circuits. Does not block export.
Port name error in VHDL	Invalid characters	Use lowercase letters and underscores only in port labels.

15.4 Vivado Issues

SYMPTOM	CAUSE	RESOLUTION
Synthesis: "Port not found"	Name mismatch	Re-export from RedByte.
Routing errors	Complex logic / loops	Simplify circuit; check for combinational loops.
Board unresponsive after program	Wrong FPGA part	Verify Vivado project targets `xc7a35t-1cpg236-1`.

16. Best Practices

DESIGN

Begin with INPUT and OUTPUT nodes to define the circuit boundary before adding internal logic. Use meaningful labels — they propagate to exported HDL port names.

VERIFICATION

Define test vectors before building complex circuits. Test edge cases: all-zero, all-one, and boundary conditions. Re-verify after every design change.

HARDWARE

Select each signal's Assign/Edit/Resolve row action, choose a compatible Basys3 resource, and use **Save assignment**. The board graphic is reference-only. Map top-level FPGA clocks to `CLK100MHZ` (W5).

EXPORT

Use the manifest for lossless re-import. Supported RedByte-generated `top.vhd` can reconstruct its graph, but it does not carry layout, authored Verify documents, mapping, or project metadata. Keep the exported ZIP alongside the Vivado project as a design record.

SUBMISSION

Fill in the student name field before exporting. Do not manually modify files inside the ZIP — any change invalidates integrity hashes.

17. Glossary

TERM	DEFINITION
Basys 3	Digilent FPGA development board (Artix-7 XC7A35T). The target hardware platform for RedByte exports.
Circuit	A collection of nodes and connections representing a digital logic design. The single source of truth.
Sequential policy	The named Verify document's execution mode, run cycles, active edge, reset behavior, source type, execution model, resolved clock/control identity, and starting level. Its frozen projection may drive generated `testbench.vhd` without becoming a portable project field.
Connection	Directional link from output port to input port. Format: `{ from: { nodeId, portName }, to: { nodeId, portName } }`.
Determinism	Identical inputs always produce identical results, across runs and machines.
Fidelity level	Import quality classification: Full, Reconstructed, or Partial.
Gate verdict	Summary judgment of whether work meets lab requirements.
LVCMOS33	3.3V low-voltage CMOS I/O standard used by the Basys 3 board.
Node	Any supported logic, sequential, boundary-I/O, board-resource, reusable, or backward-compatible circuit element.
Port	Named input or output connection point on a node.
Rising edge	Clock transition from 0 to 1. Triggers data capture in flip-flops.
Falling transition	Authored clock transition from 1 to 0. Valid stimulus that holds rising-edge state; not falling-edge-triggered capture.

TERM	DEFINITION
Stage	One of five student workflow destinations: Project, Design, Verify, Map Pins, or Export. Import / Recover is a separate utility, not a sixth stage.
Test vector	A truth table row specifying input values and expected output values.
Tick	One discrete simulation time step. All nodes evaluated once per tick in topological order.
Topological sort	Algorithm ensuring nodes are evaluated after their input dependencies.
Vivado Kit	The exported ZIP containing `top.vhd`, `top.xdc`, `testbench.vhd`, automation scripts, and documentation files.
XDC	Xilinx Design Constraints file format for pin assignments and I/O standards.

18. Appendices

Appendix A: Logic Primitive Reference

A.1 Basic Logic Gates (2-Input)

PRIMITIVE	INPUTS	OUTPUT	BEHAVIOR
AND	a, b	out	1 when both inputs are 1.
OR	a, b	out	1 when at least one input is 1.
NOT	in	out	Complement of input.
NAND	a, b	out	0 only when both inputs are 1.
NOR	a, b	out	1 only when both inputs are 0.
XOR	a, b	out	1 when inputs differ.
XNOR	a, b	out	1 when inputs match.

Registry truth: NOR and XNOR are registered runtime behaviors and current student palette entries. The runtime performs 27 direct primitive registrations plus 4 composite registrations, for 31 registry additions; palette visibility remains separately governed by the support registry.

A.2 Three-Input Gates

PRIMITIVE	INPUTS	OUTPUT	BEHAVIOR
AND3	a, b, c	out	1 when all three inputs are 1.
OR3	a, b, c	out	1 when any input is 1.
NAND3	a, b, c	out	0 when all three inputs are 1.
NOR3	a, b, c	out	0 when any input is 1.
XOR3	a, b, c	out	1 for odd number of high inputs.

A.3 Sequential Elements

PRIMITIVE	INPUTS	OUTPUTS	BEHAVIOR
Register1	D, CLK, optional EN/RST	Q	Supported one-bit register: one clock, rising-edge capture, active-high asynchronous clear, and supported active-high enable semantics. Other configurations block explicitly.
RegisterBus	D[i], CLK, optional EN/RST	Q[i]	Design-canvas bus register; Verify and Export are intentionally blocked in this stable preview because runtime/VHDL parity is incomplete.
StateBank	D[i], CLK, optional EN/RST	Q[i]	Design-canvas grouped state abstraction; Verify and Export are intentionally blocked in this stable preview because runtime/VHDL parity is incomplete.
D Flip-Flop	D, CLK, EN, RST	Q, Q̄	Captures D on rising CLK edge. RST=1 forces Q=0. EN gates capture.
T Flip-Flop	T, CLK, CLR	Q, Q̄	T=1 toggles Q on rising edge. T=0 holds.
JK Flip-Flop	J, K, CLK, CLR	Q, Q̄	J=1,K=0: set. J=0,K=1: reset. J=K=1: toggle. J=K=0: hold.

A.4 I/O and Signal Elements

PRIMITIVE	PORTS	BEHAVIOR
Switch	out	User-toggled input (0 or 1).
Lamp	in, out	Visual output indicator. Mirrors input.
INPUT	out	External input source.
OUTPUT	in	Terminal output sink.
Power Source	out	Constant 1.
Ground	out	Constant 0.
Clock	out	Periodic oscillation (configurable period).
Delay	in, out	Pass-through with configurable delay (default: 1 tick).
Wire	in, out	Simple pass-through. Output equals input.

Clock boundary: `Clock` remains a registered runtime behavior for legacy serialized projects and simulation internals, but it is not a current student-authoring palette card. Students choose Basys3 `CLK100MHZ` for FPGA clock intent; pure simulation may use an automatically injected internal clock.

Appendix B: Basys 3 Pin Reference

B.1 Slide Switches

SIGNAL	FPGA PIN
SW0	V17
SW1	V16
SW2	W16
SW3	W17
SW4–SW15	See Basys 3 Reference Manual

B.2 LEDs

SIGNAL	FPGA PIN
LD0	U16
LD1	E19
LD2	U19
LD3	V19
LD4–LD15	See Basys 3 Reference Manual

B.3 Push Buttons and Clock

SIGNAL	FPGA PIN	FUNCTION
BTNC	U18	Center button
BTNU	T18	Up button
BTNL	W19	Left button
BTNR	T17	Right button
BTND	U17	Down button
CLK100MHZ	W5	100 MHz on-board oscillator

B.4 Part Number

ATTRIBUTE	VALUE
Device	xc7a35t
Package	cpg236
Speed Grade	-1
Full Part String	`xc7a35t-1cpg236-1`

Appendix C: Generated File Specifications

FILE	ENTITY	KEY PROPERTIES
`top.vhd`	`top`	Synthesizable. Ports match Map Pins. LVCMOS33 (via XDC). No behavioral process blocks.
`top.xdc`	—	One `set_property PACKAGE_PIN` per port. `CLOCK_BUFFER_TYPE NONE` for switch/button inputs.
`testbench.vhd`	`tb_top`	Simulation only. Auto board clock uses a free-running 10 ns generator and rising-edge waits; manual/custom assigns the authored clock per vector and uses deterministic settling.

Appendix D: Import Fidelity Reference

SOURCE	FIDELITY	CIRCUIT	LAYOUT	VECTORS	PINS
`.rbproj.json` from ZIP	FULL	✓	✓	✓	✓
Structural VHDL	RECONSTRUCTED	✓ (topology)	Auto	—	—
Arbitrary behavioral/process HDL or unsupported constructs	PARTIAL / BLOCKED	Ports only or blocked	—	—	—
Supported RedByte `top.vhd`	RECONSTRUCTED	Supported graph	Auto	—	—
XDC file	—	—	—	—	✓ (if match)

RedByte

Product Manual - Version 1.3 - July 2026

Attribution: Connor Angiel